

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf\_ace7351e-c4e\agent-a70cc4ffed0c886fc.jsonl`
- SHA-256: `d49908b8487e5828049bd5b1d4ba0ce01dad6db3d8484bbdfce962be2f96a0c7`
- Source modified: `2026-07-06T22:19:35+00:00`
- Imported at: `2026-07-08T16:00:07+00:00`
- project: `wf\_ace7351e-c4e`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`

Transcript

1. user (2026-07-06T22:18:56.593Z)

Reviewing GitHub PR #52 "feat(policy): add F3 caption safety stage" (branch feat/f3-caption-safety -> main). ADR 0012 F3.
+299/-41, 4 files. Checked-out copy at C:/Users/avidu/Projects/_wt-pr52 (read there or 'git -C C:/Users/avidu/Projects/Telegram show origin/feat/f3-caption-safety:<path>').
Run repros: PYTHONPATH=C:/Users/avidu/Projects/_wt-pr52/src
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>.

WHAT F3 DOES:

- policy.py: CaptionSafetyConfig + parse_caption_safety_config (from 'security_checks' key: unsafe_caption_terms[], unsafe_caption_patterns[]);
CaptionSafetyStage (Tier 3): fail-OPEN on invalid config (pass_ + warning + invalid_caption_safety_config=True); disabled if empty;
missing caption -> pass (caption_present=False); term match = casefold substring; pattern match = re.search(pattern, caption) (case-SENSITIVE
unless user adds (?i)); regexes validated via re.compile at parse time (invalid regex -> ValueError -> fail-open).
- telethon_client.py: _evaluate_topic_preferences RENAMED to _evaluate_policy_stages, now runs [TopicPreferenceStage(), CaptionSafetyStage()]
through evaluate_pipeline() and returns the list; live+backfill wiring: 'if policy_results: reasons.extend(...); matched = policy_results[-1].passed'.
- Reads security_checks from the RAW policy config (preserved by F1's save merge).
on_new_message/backfill_user are '# pragma: no cover'.

CONFIRMED by the main reviewer: gate green (523 passed @ 95.77%); ReDoS is REAL ? the valid pattern '(a)+\$' on a 40-char caption hangs
re.search past a 6s timeout (rc=124), blocking the synchronous event loop. Feature is dormant today (no command sets security_checks).

Ground rules: READ code, cite file:line. Run repros. Report ONLY defects in THIS PR's diff. No praise. Concrete failure scenario, ranked.
Severity: high (real bug/regression/exploitable-now), medium (real but latent / host-wide-DoS-once-configurable / design), low (hygiene).

ADVERSARIALLY VERIFY this finding ? try to REFUTE it. Read cited code, run a repro with
PYTHONPATH=C:/Users/avidu/Projects/_wt-pr52/src
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe.
CONFIRMED only if the defect genuinely ships in THIS PR's diff and produces the described wrong behaviour. REFUTED if handled / can't occur /
pre-existing / out-of-scope-and-disclosed. PLAUSIBLE if unsettled. Corrected severity (recall: dormant-but-real host-DoS = medium, hygiene = low).
FINDING (wiring_parse): Caption safety term matching is unanchored casefold substring, so short unsafe terms produce false-positive rejections on innocuous captions
severity low | src/tg_video_filter/policy.py:281

summary: CaptionSafetyStage matches a term with `term.casefold()` in haystack` (plain substring), so a configured term like `ass` rejects captions containing `classic`, `passage`, `embassy`, etc. No word-boundary option exists; operators can only escalate to regex patterns (which carry the confirmed ReDoS risk).

scenario: With `security\_checks.unsafe\_caption\_terms=["ass"]`, a caption `"classic passage"` is rejected: repro returns `passed=False reason='unsafe caption term matched: ass'`. Since `matched` then becomes False, a legitimate video is dropped from delivery. The feature is dormant today (no command sets security_checks), so impact is latent, but any operator using short terms will silently over-block.

evidence: policy.py:280-281 `haystack = caption.casefold()` then `for term in config.unsafe\_terms: if term.casefold() in haystack: -> fail. Repro: substring `ass` in `classic passage` -> `passed=False | unsafe caption term matched: ass`.

rec: Document the substring semantics explicitly for operators, and/or offer word-boundary matching (e.g. an opt-in `bterm` mode) so short terms don't over-match. At minimum note in the config docs that terms are unanchored substrings.

2. assistant (2026-07-06T22:18:59.201Z)

I'll verify this finding by reading the cited code and running a repro.

3. user (2026-07-06T22:19:00.028Z)

```
255         except (json.JSONDecodeError, ValueError) as exc:
256             log.warning(
257                 "caption safety disabled due to invalid config policy_id=%s",
258                 ctx.policy.id,
259                 exc_info=True,
260             )
261             return StageResult.pass_(
262                 self.stage_name,
263                 f"invalid caption safety config ignored: {exc}",
264                 invalid_caption_safety_config=True,
265             )
266         if not config.enabled:
267             return StageResult.pass_(self.stage_name, "caption safety disabled")
268
269         caption = (ctx.content_item.text or "").strip()
270         if not caption:
271             return StageResult.pass_(
272                 self.stage_name,
273                 "no caption to scan",
274                 caption_present=False,
275             )
276
277         haystack = caption.casefold()
278         for term in config.unsafe_terms:
279             if term.casefold() in haystack:
280                 return StageResult.fail(
281                     self.stage_name,
282                     f"unsafe caption term matched: {term}",
283                     caption_present=True,
284                     match_type="term",
285                     matched_caption_term=term,
286                 )
287
288         for pattern in config.unsafe_patterns:
289             if re.search(pattern, caption):
290                 return StageResult.fail(
291                     self.stage_name,
292                     "unsafe caption pattern matched",
293                     caption_present=True,
294                     match_type="pattern",
295                     matched_caption_pattern=pattern,
296                 )
297
```

```

298         return StageResult.pass_(
299             self.stage_name,
300             "caption passed safety checks",
301             caption_present=True,
302             checked_terms=len(config.unsafe_terms),
303             checked_patterns=len(config.unsafe_patterns),
304         )
305
306
307     async def evaluate_pipeline(
308         ctx: PolicyContext,
309         stages: list[PolicyStage],
310     ) -> tuple[bool, list[StageResult]]:
311         """Evaluate stages in order and short-circuit on the first failure."""
312         results: list[StageResult] = []
313         for stage in stages:
314             result = await stage.evaluate(ctx)

```

4. assistant (2026-07-06T22:19:02.397Z)

Confirmed the substring match at policy.py:277-279. Now let me verify the config parsing to confirm terms have no word-boundary option, and run the repro.

5. user (2026-07-06T22:19:04.726Z)

```

82: class CaptionSafetyConfig:
85:     unsafe_terms: tuple[str, ...] = ()
90:     return bool(self.unsafe_terms or self.unsafe_patterns)
114: def parse_caption_safety_config(config: dict[str, object]) -> CaptionSafetyConfig:
118:     ``{"security_checks":{"unsafe_caption_terms":[],"unsafe_caption_patterns":[]}}``.
126:     unsafe_terms = _string_tuple(
127:         raw_security.get("unsafe_caption_terms"),
128:         field_name="security_checks.unsafe_caption_terms",
143:         unsafe_terms=unsafe_terms,
278:         for term in config.unsafe_terms:
302:             checked_terms=len(config.unsafe_terms),

```

6. user (2026-07-06T22:19:07.137Z)

```

80
81     @dataclass(frozen=True)
82     class CaptionSafetyConfig:
83         """Parsed ADR 0012 caption safety config."""
84
85         unsafe_terms: tuple[str, ...] = ()
86         unsafe_patterns: tuple[str, ...] = ()
87
88         @property
89         def enabled(self) -> bool:
90             return bool(self.unsafe_terms or self.unsafe_patterns)
91
92
93     def parse_topic_preferences(config: dict[str, object]) -> TopicPreferences:
94         """Parse ``topics`` from a raw policy config object."""
95         raw_topics = config.get("topics")
96         if raw_topics is None:
97             return TopicPreferences()
98         if not isinstance(raw_topics, dict):
99             raise ValueError("topics must be a JSON object")
100         include = _string_tuple(raw_topics.get("include"), field_name="topics.include")
101         exclude = _string_tuple(raw_topics.get("exclude"), field_name="topics.exclude")
102         if not include and not exclude:
103             return TopicPreferences()
104         mode = raw_topics.get("mode", "any")

```

```

105         if mode not in ("any", "all"):
106             raise ValueError("topics.mode must be 'any' or 'all'")
107         return TopicPreferences(
108             include=include,
109             exclude=exclude,
110             mode=str(mode),
111         )
112
113
114     def parse_caption_safety_config(config: dict[str, object]) -> CaptionSafetyConfig:
115         """Parse caption safety settings from ``security_checks``.
116
117         Expected shape:
118         ``{"security_checks":{"unsafe_caption_terms":[],"unsafe_caption_patterns":[]}}``.
119         """
120         raw_security = config.get("security_checks")
121         if raw_security is None:
122             return CaptionSafetyConfig()
123         if not isinstance(raw_security, dict):
124             raise ValueError("security_checks must be a JSON object")
125
126         unsafe_terms = _string_tuple(
127             raw_security.get("unsafe_caption_terms"),
128             field_name="security_checks.unsafe_caption_terms",
129         )
130         unsafe_patterns = _string_tuple(
131             raw_security.get("unsafe_caption_patterns"),
132             field_name="security_checks.unsafe_caption_patterns",
133         )
134         for pattern in unsafe_patterns:
135             try:
136                 re.compile(pattern)
137             except re.error as exc:
138                 raise ValueError(
139                     "security_checks.unsafe_caption_patterns contains invalid regex "
140                     f"{pattern!r}: {exc}"
141                 ) from exc
142         return CaptionSafetyConfig(
143             unsafe_terms=unsafe_terms,
144             unsafe_patterns=unsafe_patterns,
145         )
146
147
148     def load_topic_preferences(policy: Policy) -> TopicPreferences:
149         """Load topic preferences from ``policy.config_json``."""

```

7. assistant (2026-07-06T22:19:09.485Z)

Confirmed: no word-boundary option exists. Terms are matched as plain casefold substrings, patterns are the only escalation. Now let me run the repro to confirm the described behavior.

8. user (2026-07-06T22:19:13.975Z)

```

enabled: True terms: ('ass',)
MATCH -> passed=False | unsafe caption term matched: ass

```

9. assistant (2026-07-06T22:19:19.275Z)

The repro confirms the parse and the substring logic. Let me verify the full stage path end-to-end (that `passed=False` truly results, exercising the actual `CaptionSafetyStage.evaluate`) to be thorough about the "matched becomes False -> video dropped" claim.

10. user (2026-07-06T22:19:20.192Z)

```

230         )
231     return StageResult.pass_(
232         self.stage_name,
233         "matched all include topics",
234         matched_include_topics=list(matched_includes),
235     )
236
237
238     def _topic_haystack(ctx: PolicyContext) -> str:
239         parts = [ctx.source.title or "", ctx.content_item.text or ""]
240         return "\n".join(parts).casefold()
241
242
243     def _matched_topics(topics: tuple[str, ...], haystack: str) -> tuple[str, ...]:
244         return tuple(topic for topic in topics if topic.casefold() in haystack)
245
246
247     class CaptionSafetyStage:
248         """ADR 0012 Tier 3 captured-caption term and pattern scanning."""
249
250         stage_name = "caption-safety"
251
252         async def evaluate(self, ctx: PolicyContext) -> StageResult:
253             try:
254                 config = load_caption_safety_config(ctx.policy)
255             except (json.JSONDecodeError, ValueError) as exc:
256                 log.warning(
257                     "caption safety disabled due to invalid config policy_id=%s",

```

11. assistant (2026-07-06T22:19:29.305Z)

The finding is fully verified by code and repro. Let me assess severity.

The finding is factually correct:

- policy.py:277-279 does `haystack = caption.casefold()` then `if term.casefold() in haystack` ? plain unanchored substring, no word boundary.
- No word-boundary option is parsed (policy.py:82-90, 114-145). Terms are only ever substrings; the only escalation is regex patterns (which carry the confirmed ReDoS risk).
- Repro confirms `ass` matches `classic passage` -> fail -> `matched` becomes False -> legitimate video dropped.

This is genuine and ships in this PR's diff. On severity: the reported `low` is appropriate. Substring matching is a real over-matching design characteristic, but (a) the feature is dormant today (no command sets `security\_checks`), and (b) even once enabled, whether substring matching is "wrong" depends on operator intent ? substring matching is a defensible default for a safety blocklist, and the false-positive requires the operator to choose short/common terms. It is over-blocking (fail-closed on legit content), not a security hole or host-DoS. That is design/hygiene territory. Dormant + design/UX papercut = `low`.

12. user (2026-07-06T22:19:35.386Z)

Structured output provided successfully